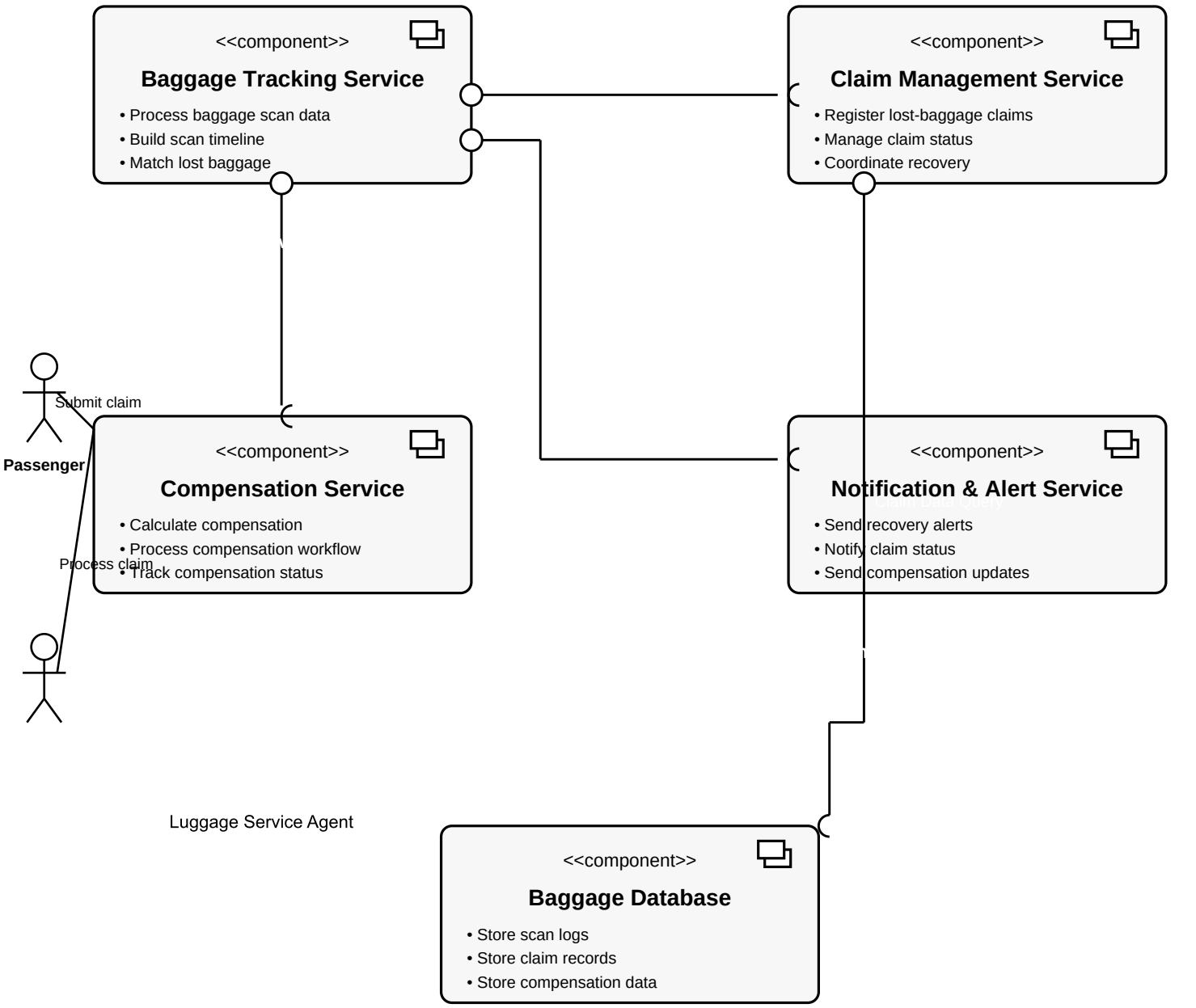
Airport Lost Luggage Claim & Tracking Portal

UML Component Diagram

Selected Architecture: Microservices



LEGEND

- Provided Interface (Ball) — service offered by a component
- Required Interface (Socket) — service needed by a component
- Interface connection
- Dependency / usage line

Architecture Selection & Justification

Airport Lost Luggage Claim & Tracking Portal

Architecture Selection:

We chose Microservices Architecture for the Airport Lost Luggage Claim & Tracking Portal.

1. Architectural Choice — Reason 1

The portal contains separate functions such as baggage scan tracking, lost-baggage claim management, compensation processing, and recovery notifications. Microservices separates these responsibilities into focused services with clear interfaces.

2. Architectural Choice — Reason 2

Airport baggage activity can increase sharply during peak travel periods. Individual services such as tracking or claim management can be scaled independently instead of scaling the complete application.

3. Security Advantage

Service boundaries allow access control to be applied to individual operations. Claim and compensation functions can be restricted to authorized passengers, baggage service agents, and trusted service-to-service requests.

4. Performance Benefit

The baggage tracking service can process scan-history requests independently of compensation and notification work. This supports the supplied requirement that the tracking timeline API return complete multi-leg scan history in under 200 ms.

Brief Architecture Evaluation

Architecture	Strength	Limitation for this scenario
Layered	Clear separation of concerns	Harder to scale individual functions
Microservices	Independent scaling and fault isolation	More operational complexity
Client-Server	Centralized control and consistency	Central server can become a bottleneck

Prepared from the supplied Lab 3 instructions and Airport Lost Luggage problem statement.